Exercise 1

- Static vs non-static fields
  - they belong to the class not each individual object
  - meanwhile non-static can be different for any object of a class
- Reference vs primitive data types
  - primitive data types are provided by a programming language most of the times
  - meanwhile r.d.t. are pointers which do not hold their values but are also used with objects
- Overriding vs overloading
  - when the method signature (name and parameters) are the same in the superclass and the child class.
  - meanwhile overloading happens when two or more methods in the same class have the same name but different parameters
- Interface vs Class
  - group of related properties and methods that describe an object, no initialization or implementation.
  - meanwhile a class is a blueprint from which we can create objects that share the same configuration properties and methods

- Interface vs abstract
  - permits you to state functionality but not to implement it, a class can implement multiple interfaces
  - meanwhile an abstract permits you to make functions that subclasses can implement or override, a class can extend only one abstract
- Threads vs processes
  - subset of process, smallest part of a process that can be executed concurrently with other parts (threads) of the process
  - program execution is referred as the process

- multithreading: multiple threads, is the ability of the cpu to execute multiple threads independently at the same time but sharing the process resources simultaneously
- states of thread: NEW - not started
  RUNNABLE - executing
  BLOCKED - waiting for a monitor lock
  WAITING - waiting for another thread to finish
  TIME WAITING - same as above but specified
  TERMINATED - finished its state

~~+ sync~~: it ensures ~~that two or more concurrent processes or threads do not simultaneously execute some particular program, critical section; monitor model~~.

+ sync: capability to control the access of multiple threads to any shared resource, bcs. this can produce inconsistent results; we need sync for reliable comms. between threads

monitor model: sync construct that allows threads to have mutual exclusion an the ability to wait for a certain condition to become false

Exercise IV

EXECUTOR SERVICE
- asynchronous execution mechanism which is capable of executing tasks in the background

ForkJoinPool
- similar to ?
- work stealing strategy: every time a thread has to wait for some result, the thread removes the current thread from the q. and executes some other task, ready to run

BLOCKINGQUEUE
- four forms
- one throws an exception: add(e), remove(), element()
- two; special value offer(), poll(), peek()
- block current thread indefinitely until the operation can succeed: put(e), take()
- fourth blocks for a given maximum time limit before giving up: offer(e, time, unit); poll(time, unit)

SEMAPHORE
- gives access to resources through the use of a counter; if the counter is "> 0" access is allowed, otherwise not; the counter counts permits that allow access, thus to access a resource a permit must be granted to a thread from the semaphore
- S.count > 0 => thread has permit => S.count decremented
- thread blocked until permit otherwise
- if thread no longer needs access to the shared resource releases permit => S.count incremented
- next thread

COUNTDOWNLATCH

- initialized with a given count
- decremented by calls of countDown method
- used to thread block until other threads have completed a given task

CyclicBarrier

- sync mechanism
- barriers that all threads must wait until all threads reach it, before all can continue

Lock

- sync mechanism; just like synchronized blocks
- more flexible and sophisticated that?
- ReadWriteLock
  - threads (many) reach a resource but only one to write it at a time
  - read lock - -||- for reading
  - write lock - no threads read or write => one thread at a time can lock the lock for writing

ATOMIC VARIABLES

- heavy use of compare and swap
- faster than locks
- AtomicBoolean - read or written automatically
- AtomicInteger
- .
- .
- .

Map 3 - interfaces *
- downcasting
- instance of
- to String *
- inheritance *
- packages *
- overloading *
- polymorphism *
- abstract *

this 64

Map 4 - downcasting
- class Object
- to String *
- Packages
- access modifiers
- exceptions
- finally clause
- enhanced for
- java generics *

Map 5 - generics recap *
- generics class usage
- autoboxing
- raw types
- generic arrays
- bounds
- wildcards *
- bounded wildcards *
- wildcards upper / lower bound *
- java collection
- Comparable and comparators

Map 6 - java.io
- inputstream / outputstream
- reader
- writer
- standard streams *
- BufferReader/Writer *
- printwriter
- reading from keyboard
- java.nio
- channels and buffers *
- selectors

Map 7 - java serialization and functional programming
- Anonymous inner class
- Lambda expressions (you skipped it)
- java.util.function *
- streams *
- obtaining a stream from a collection

Map 8 - Stream min max
- Stream reduce
- Stream infinite
- Stream finding and matching

Map 9 - threads *
- multithreading
- producer
- java.util.concurrent *
- google drive